

Creating Microservices for account and loan

In this hands on exercises, we will create two microservices for a bank. One microservice for handling accounts and one for handling loans.

Each microservice will be a specific independent Spring RESTful Webservice maven project having it's own pom.xml. The only difference is that, instead of having both account and loan as a single application, it is split into two different applications. These webservices will be a simple service without any backend connectivity.

Follow steps below to implement the two microservices:

Account Microservice

- Create folder with employee id in D: drive
- Create folder named 'microservices' in the new folder created in previous step. This folder will contain all the sample projects that we will create for learning microservices.
- Open <https://start.spring.io/> in browser
- Enter form field values as specified below:
 - **Group:** com.cognizant
 - **Artifact:** account
- Select the following modules
 - Developer Tools > Spring Boot DevTools
 - Web > Spring Web
- Click generate and download the zip file
- Extract 'account' folder from the zip and place this folder in the 'microservices' folder created earlier
- Open command prompt in account folder and build using mvn clean package command
- Import this project in Eclipse and implement a controller method for getting account details based on account number. Refer specification below:
 - Method: GET
 - Endpoint: /accounts/{number}
 - Sample Response. Just a dummy response without any backend connectivity.

```
{ number: "00987987973432", type: "savings", balance: 234343 }
```

- Launch by running the application class and test the service in browser

Loan Microservice

- Follow similar steps specified for Account Microservice and implement a service API to get loan account details
 - Method: GET
 - Endpoint: /loans/{number}
 - Sample Response. Just a dummy response without any backend connectivity.

```
{ number: "H00987987972342", type: "car", loan: 400000, emi: 3258, tenure: 18 }
```

- Launching this application by having account service already running
- This launch will fail with error that the bind address is already in use
- The reason is that each one of the service is launched with default port number as 8080. Account service is already using this port and it is not available for loan service.
- Include "server.port" property with value 8081 and try launching the application
- Test the service with 8081 port

Now we have two microservices running on different ports.

NOTE: The console window of Eclipse will have both the service console running. To switch between different consoles use the monitor icon within the console view.

Week 4 – Creating Microservices for Account and Loan

Objective

To create two independent Spring Boot REST microservices:

- Account Microservice
- Loan Microservice

Each service runs independently on a different port and returns dummy JSON data without backend connectivity.

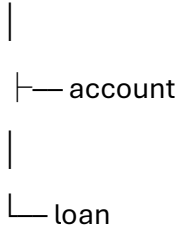
Technologies Used

- Java 17
- Spring Boot 4
- Spring Web

- Spring Boot DevTools
- Maven
- IntelliJ IDEA

Project Structure

microservices



Account Microservice

Endpoint

GET /accounts/{number}

URL

http://localhost:8080/accounts/00987987973432

Sample Response

```
{
  "number": "00987987973432",
  "type": "savings",
  "balance": 234343.0
}
```

Loan Microservice

Endpoint

GET /loans/{number}

URL

http://localhost:8081/loans/H00987987972342

Sample Response

```
{  
  "number": "H00987987972342",  
  "type": "car",  
  "loan": 400000.0,  
  "emi": 3258,  
  "tenure": 18  
}
```

Port Configuration

By default, Spring Boot applications run on **8080**.

The Loan microservice was configured to use:

`server.port=8081`

to avoid port conflicts.

Result

Successfully created and tested two independent Spring Boot microservices.

- Account Service running on Port **8080**
- Loan Service running on Port **8081**

Both REST APIs returned the expected JSON responses.

p



Project

☐ Gradle - Groovy
☐ Gradle - Kotlin
☒ Maven

Language

☒ Java
☐ Kotlin
☐ Groovy

Spring Boot

☐ 4.1.1 (SNAPSHOT)
☒ 4.1.0
☐ 4.0.8 (SNAPSHOT)
☐ 4.0.7

Project Metadata

Group

com.cognizant

Artifact

account

Package name

com.cognizant.account

Packaging

☒ Jar
☐ War

Configuration

☒ Properties
☐ YAML

Java

☐ 26
☐ 25
☐ 21
☒ 17

Dependencies

ADD DEPENDENCIES... CTRL+B

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Boot DevTools

DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

GENERATE CTRL+G

EXPLORE CTRL+SPACE

...

Project

account

main

java

com.cognizant.account

AccountApplication

resources

test

target

gitattributes

gitignore

HELP.md

mvnw

mvnw.cmd

pom.xml

Search Everywhere F1 Ctrl+Shift+A

Go to File Ctrl+P

Recent Files Ctrl+E

Navigation Bar Ctrl+Shift+;

Drop files here to open them

Maven

Profiles

m account

Terminal

Local

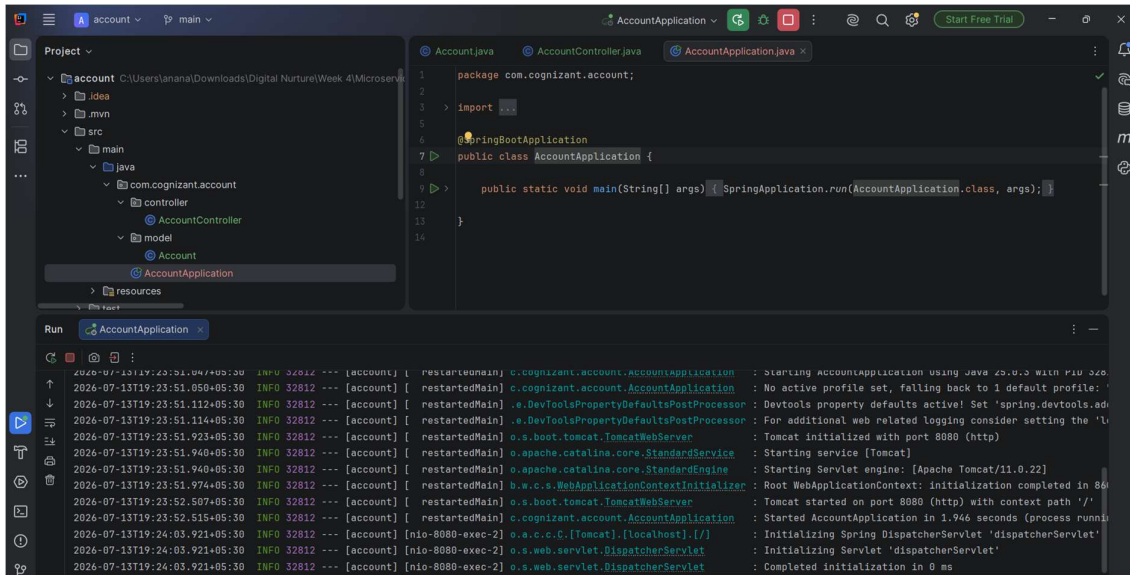
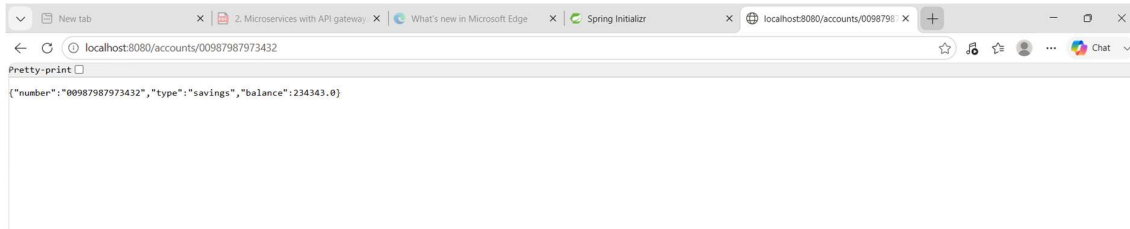
```
[INFO] Replacing main artifact C:\Users\anana\Downloads\Digital Nuture\Week 4\Microservices with Spring Boot 3 and Spring Cloud\Creating Microservices for account and loan\microservices\account\target\account-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/...
[INFO] The original artifact has been renamed to C:\Users\anana\Downloads\Digital Nuture\Week 4\Microservices with Spring Boot 3 and Spring Cloud\Creating Microservices for account and loan\microservices\account\target\account-0.0.1-SNAPSHOT.jar.original
[INFO] BUILD SUCCESS
[INFO] Total time: 23.872 s
[INFO] Finished at: 2026-07-13T19:21:07+05:30
```

35°C Mostly cloudy

Search

ENG IN

1921 13-07-2026



Spring Initializr

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

☐ 4.1.1 (SNAPSHOT)

☒ 4.1.0

☐ 4.0.8 (SNAPSHOT)

☐ 4.0.7

Project Metadata

Group

com.cognizant

Artifact

loan

Package name

com.cognizant.loan

Packaging

☒ Jar

☐ War

Configuration

☒ Properties

☐ YAML

Java

☐ 26

☐ 25

☐ 21

☒ 17

Language

☒ Java

☐ Kotlin

☐ Groovy

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

—

Spring Boot DevTools

DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

—

GENERATE CTRL + G

EXPLORE CTRL + SPACE

...

Project

loan

main

src

main

java

com.cognizant.loan

controller

LoanController

model

Loan

LoanApplication

resources

static

templates

application.properties

test

target

Loan.java

LoanController.java

application.properties

LoanApplication.java

```
1 package com.cognizant.loan;
2
3 > import ...
4
5
6 @SpringBootApplication
7 public class LoanApplication {
8
9     public static void main(String[] args) { SpringApplication.run(LoanApplication.class, args); }
10
11
12 }
13
14
```

Run

LoanApplication

2026-07-13T19:34:06.849+05:30 INFO 29724 --- [loan] [restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]

2026-07-13T19:34:06.987+05:30 INFO 29724 --- [loan] [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 870 ms

2026-07-13T19:34:07.312+05:30 INFO 29724 --- [loan] [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8081 (http) with context path '/'

2026-07-13T19:34:07.320+05:30 INFO 29724 --- [loan] [restartedMain] com.cognizant.loan.LoanApplication : Started LoanApplication in 1.761 seconds (process running for ...)

2026-07-13T19:34:19.106+05:30 INFO 29724 --- [loan] [nio-8081-exec-1] o.a.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'

2026-07-13T19:34:19.109+05:30 INFO 29724 --- [loan] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'

2026-07-13T19:34:19.111+05:30 INFO 29724 --- [loan] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms

localhost:8081/loans/H00987987972342

Pretty-print

```
{ "number": "H00987987972342", "type": "car", "loan": 400000.0, "emi": 13258, "tenure": 18 }
```